claude-windows / 166d7059-d8ac-4fdd-a8c0-072cdaa7eccd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\166d7059-d8ac-4fdd-a8c0-072cdaa7eccd\subagents\workflows\wf_66c34d9d-057\agent-a907a2223ebec88d5.jsonl`
- SHA-256: `7ecf1b2384d4f3eed6c14af182b86c54f8a1594d81b7e1f317e9a47484b968ff`
- Source modified: `2026-06-18T05:35:27+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_66c34d9d-057`
- session_id: `166d7059-d8ac-4fdd-a8c0-072cdaa7eccd`

Transcript

1. user (2026-06-18T05:31:43.301Z)

```
WORKTREE (cd here; code is origin/master f5f339a): C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/verify-n13
Run python as:  cd "C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/verify-n13" && PYTHONPATH=. python <your_script>
GOLDEN/manifest/features dir (ABSOLUTE ? pass this, the worktree output/ is empty):
C:/Users/avidu/Projects/badminton-highlight-indexer/output

REPRODUCE RECIPE (use the public API; write your own short script to scratch/lens_<name>.py):
  import os
  from dataclasses import replace
  from backend.eval import rally_seg_eval as rse, eval_stats
  from backend.eval.windowing import SERVED
  from backend.eval.tolerance_metrics import over_seg_guard
  OUT = "C:/Users/avidu/Projects/badminton-highlight-indexer/output"
  golden = [g for g in rse.load_golden(os.path.join(OUT,"human_lovo_manifest.json"), OUT)
            if g["gts"] and os.path.exists(g["trajectory"])]   # expect 13
  baseline  = SERVED                                          # all windowing knobs OFF
  cap12     = replace(SERVED, max_window_duration=12.0)
  cap6      = replace(SERVED, max_window_duration=6.0)
  milestone = replace(SERVED, max_window_duration=6.0, inactivity_end=1.5,
                      inactivity_rally_thresh=0.20, inactivity_min_density=0.30)
  rows = lambda p: {r["name"]: r for r in rse.evaluate(golden, p)["rows"]}  # each row: tol_f1,
      # f1, merge_rate, split_rate, merge_count, split_count, num_pred, num_gt, name,
dend_abs_median
  # paired sign-test on a metric: eval_stats.paired_delta_ci([base[n][k] for n in
names],[cand[n][k] for n in names])
  #   returns {mean_delta, ci_low, ci_high, ci_excludes_zero,
sign_test:{wins,losses,ties,p_value}}
  # over-seg guard: over_seg_guard(list(base_rows.values()), list(cand_rows.values()))
  #   returns {passes, strict_passes, base_net, cand_net, net_delta, no_merge_rate_regress,
merge_rate_regress:[...]}
  # NOTE: rse.evaluate may print "R5 blob-fallback: forced ..." lines for the blob clip ? that
is the
  #   R5 logger and is HARMLESS/irrelevant here (the milestone does NOT enable blob_fallback).
Ignore it.

THE CLAIM TO FALSIFY: "The R1 milestone (W1 cap=6 + R4 inactivity-end=1.5/rally_thresh=0.20/
min_density=0.30, no W2, no R5) still clears the net-over-seg promotion bar at n=13 ? a
SIGNIFICANT
R2 tolF1 win over baseline AND no honest over-seg regression ? so config-flip PR #204 still
stands."

MY (the orchestrator's) measured n=13 numbers, for you to INDEPENDENTLY confirm or REFUTE ? do
```

```
NOT
trust them, recompute:
  - baseline->milestone tolF1: ?+0.2222 [+0.1522,+0.2859], sign 12/0/1, p=0.0005
  - cap12->cap6 tolF1:         ?+0.1099 [+0.0703,+0.1526], sign 12/0/1, p=0.0005
  - cap6->+R4 tolF1:           ?+0.0397 [+0.0145,+0.0651], sign 9/1/3,  p=0.0215   (ONE loss ?
find it)
  - mean merge_rate baseline->milestone: 0.651 -> 0.161; over_seg_guard NET PASS, no merge_rate
regression
  - new clips: GX010137 tolF1 0.448->0.685, GX010141 0.333->0.431 (both improve)


LENS = leave-one-video-out robustness (adversarial). For the baseline->milestone tolF1
comparison, recompute the paired delta + sign-test 13 times, each time DROPPING one clip (n=12).
Does the
verdict (CI excludes 0 AND sign p<0.05) survive EVERY single drop? Report the WORST-case drop
(which clip,
and the resulting mean ? + sign + p). Pay special attention to dropping the 2 NEW clips
(GX010137,
GX010141) and the blob (mahadevpura_1). Is the win driven by any single clip? Conclude whether
the
milestone verdict is robust to LOO at n=13.

Return ONLY the structured verdict. Be precise with numbers.
```

2. assistant (2026-06-18T05:31:46.510Z)

```
I'll verify this claim by independently recomputing the leave-one-video-out robustness analysis.
Let me start by exploring the worktree and understanding the API.
```

3. user (2026-06-18T05:31:48.614Z)

```
f5f339a fix(eval): fusion loaders skip windowing-only files on a mixed golden corpus (#210)
db96ddc docs: capture external due-diligence review inputs (research, pick-up-later) (#207)
af5b3e6 docs(rally): tracked doc for closing the detection accuracy gap (golden-driven, per-
video) (#211)
__init__.py
__pycache__
ablation.py
annotations.py
audio
batch.py
calibrate_local.py
calibrate_wasb.py
calibration.py
classifier.py
distill_local.py
eval_partial_labels.py
eval_stats.py
experiment.py
export_wasb_segments.py
fusion_audio.py
fusion_compare.py
fusion_golden.py
gemini_refine.py
golden_fixtures.py
gt_loader.py
harness.py
metrics.py
per_user_eval.py
per_user_gate.py
per_user_regression.py
promotion.py
rally_seg_eval.py
rally_seq_proto.py
regression.py
```

```
score_calibration.py
segmentation_metrics.py
serve_contrast.py
served_gate_a.py
splits.py
tolerance_metrics.py
training.py
windowing.py
```

4. user (2026-06-18T05:31:49.239Z)

```
Badminton_BXH_2.rallies.csv
Badminton_BXH_2_golden_features.json
GX010125_highlights.mp4
GX010125_highlights_min5.mp4
GX010125_run
GX010128.rallies.csv
GX010128_golden_features.json
GX010137.rallies.csv
GX010137_golden_features.json
GX010141.rallies.csv
GX010141_golden_features.json
GX030094.rallies.csv
GX030094_golden_features.json
KushagraAviHighlight-Trailor.mp4
MahadevpuraSingles_highlights.mp4
REAL_FOOTAGE_VALIDATION_REPORT.md
ab_experiment_n6.json
adarsh_avi_full_vs_human.csv
adarsh_avi_singles_golden_features.json
adarsh_avi_vs_human.csv
adarsh_baseline_gated.csv
adarsh_baseline_segments.csv
adarsh_gemini_fullvideo.csv
adarsh_gemini_refined.csv
adarsh_gemini_smoke.csv
adarsh_min5s_segments.csv
adarsh_tuned_segments.csv
badminton_indexer.db
chunks
e2e_reel.mp4
```

5. assistant (2026-06-18T05:31:52.230Z)

```
Let me look at the key API functions before writing my verification script.
```

6. user (2026-06-18T05:31:54.627Z)

```
101:def load_golden(manifest_path: str = DEFAULT_MANIFEST,
147:def evaluate(golden: List[Dict[str, Any]], preset: WindowingPreset,
```

7. user (2026-06-18T05:31:55.782Z)

```
90:def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
```

8. user (2026-06-18T05:31:56.961Z)

```
6:deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED pipeline
14:SERVED preset is **single-sourced from the same Pydantic config defaults production reads**
20:The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
38:    "SERVED",
103:    """The SERVED windowing, lifted from the SAME Pydantic config defaults production reads.
111:    point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
```

```
132:SERVED: WindowingPreset = _served_preset_from_defaults()
136:#: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
143:PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
147:    """The SERVED windowing a *specific* live config serves (overrides honoured).
```

9. user (2026-06-18T05:32:01.187Z)

```
90      def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
91                          confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
92                          eps: float = 0.0) -> Dict[str, Any]:
93          """Paired (by fold) B ? A summary: mean delta + bootstrap CI + the sign test (C1).
94
95          ``a_values``/``b_values`` are per-fold held-out scores for variants A and B, aligned
by
96          fold (same video order). A promotion is honest when the delta CI clears 0 *and* the
sign
97          test agrees ? not when a single mean ticked up.
98          """
99          n = min(len(a_values), len(b_values))
100         deltas = [float(b_values[i]) - float(a_values[i]) for i in range(n)]
101         summary = mean_with_ci(deltas, confidence, n_boot, seed)
102         return {
103             "mean_delta": summary["mean"],
104             "ci_low": summary["ci_low"],
105             "ci_high": summary["ci_high"],
106             "n": float(n),
107             "ci_excludes_zero": (summary["ci_low"] > 0.0) or (summary["ci_high"] < 0.0),
108             "sign_test": paired_sign_test(deltas, eps),
109         }
110
111
112     def grouped_folds(groups: Sequence[Any]) -> List[Tuple[List[int], List[int]]]:
113         """Leave-one-group-out splits ``(train_idx, test_idx)`` ? group by match, the honest
unit.
114
115         The field's name for "LOVO by match" is leave-one-group-out (LOGO). Group order is
116         first-seen (deterministic). Groups with no complement (a single group) yield no
fold.
117         """
118         seen: List[Any] = []
119         for g in groups:
120             if g not in seen:
121                 seen.append(g)
122         n = len(groups)
123         folds: List[Tuple[List[int], List[int]]] = []
124         for g in seen:
125             test = [i for i in range(n) if groups[i] == g]
126             train = [i for i in range(n) if groups[i] != g]
127             if test and train:
128                 folds.append((train, test))
129         return folds
130
131
132     def out_of_fold_predictions(groups: Sequence[Any], fit_predict: FitPredict) ->
List[Optional[Any]]:
133         """Out-of-fold predictions for honest stacking (C3).
134
135         For each leave-one-group-out fold, ``fit_predict(train_idx, test_idx)`` trains on
the
136         other groups and returns predictions for ``test_idx`` (in that order); results are
137         scattered back to a list aligned to the input items. So a *learned* producer's
outputs,
138         when used as arbiter features, never include in-sample predictions ? the leakage
path
139         that inflates measured lift on a tiny corpus. Items in a singleton group stay
```

```
``None``
140         (needs >= 2 groups to predict).
141         """
142         n = len(groups)
143         oof: List[Optional[Any]] = [None] * n
144         for train_idx, test_idx in grouped_folds(groups):
145             preds = fit_predict(train_idx, test_idx)
146             for pos, ti in enumerate(test_idx):
147                 if pos < len(preds):
148                     oof[ti] = preds[pos]
149         return oof
150
```

10. user (2026-06-18T05:32:02.016Z)

```
147     def evaluate(golden: List[Dict[str, Any]], preset: WindowingPreset,
148                  iou: float = DEFAULT_IOU, min_crossings: int = 0,
149                  tau_start: Optional[float] = None, tau_end: Optional[float] = None) ->
Dict[str, Any]:
150         """Score every golden video under ``preset`` (+ optional net-crossings gate); return
per-video
151         rows + the aggregate. Each row carries both the tIoU set and the R2 tolerance-match
block."""
152         rows: List[Dict[str, Any]] = []
153         for v in golden:
154             if not v["gts"] or not os.path.exists(v["trajectory"]):
155                 continue
156             preds = windows_for(v, preset, min_crossings)
157             row = score_intervals(preds, v["gts"], iou, tau_start, tau_end)
158             row["name"] = v["name"]
159             rows.append(row)
160         return {"preset": preset.to_dict(), "iou": iou, "min_crossings": min_crossings,
161                 "rows": rows, "aggregate": aggregate(rows)}
162
163
164     def compare(golden: List[Dict[str, Any]], base: WindowingPreset, cand: WindowingPreset,
165                 iou: float = DEFAULT_IOU, base_min_crossings: int = 0,
166                 cand_min_crossings: int = 0,
167                 tau_start: Optional[float] = None, tau_end: Optional[float] = None,
168                 guard_weight_merge: Optional[float] = None,
169                 guard_weight_split: Optional[float] = None,
170                 guard_use_rates: bool = True) -> Dict[str, Any]:
171         """Baseline vs candidate (windowing and/or net-crossings gate): per-video paired ?
(+ CI +
172         sign-test) on both tIoU-F1 and the R2 ``tol_f1`` ? the per-tier 'number'. Also
reports ? on the
173         over-segmentation metrics AND the **R1 net over-seg promotion guard**
(``over_seg_guard``):
174         the honest promote/block verdict for cand-over-base, with the strict per-video rule
reported
175         alongside so the refinement is auditable."""
176         from backend.eval import tolerance_metrics as tm
177         b = evaluate(golden, base, iou, base_min_crossings, tau_start, tau_end)
178         c = evaluate(golden, cand, iou, cand_min_crossings, tau_start, tau_end)
179         by_name_b = {r["name"]: r for r in b["rows"]}
180         paired = [(by_name_b[r["name"]], r) for r in c["rows"] if r["name"] in by_name_b]
181         delta: Dict[str, Any] = {}
182         for k in ("f1", "f1@0.25", "tol_f1", "merge_rate", "split_rate",
"segment_count_ratio"):
183             delta[k] = eval_stats.paired_delta_ci([rb[k] for rb, _ in paired],
184                                                   [rc[k] for _, rc in paired])
185         gkw: Dict[str, Any] = {"use_rates": guard_use_rates}
186         if guard_weight_merge is not None:
187             gkw["weight_merge"] = guard_weight_merge
188         if guard_weight_split is not None:
```

```
189             gkw["weight_split"] = guard_weight_split
190         guard = tm.over_seg_guard([rb for rb, _ in paired], [rc for _, rc in paired], **gkw)
191         return {"base": b, "cand": c, "n_paired": len(paired), "delta": delta,
"over_seg_guard": guard}
192
193
194     # ---------------------------------------------------------------------- #
195     # Reporting
196     # ---------------------------------------------------------------------- #
197     def _ci(m: Optional[Dict[str, Any]]) -> str:
198         return "?" if not m else f"{m['mean']:.3f} [{m['ci_low']:.3f},{m['ci_high']:.3f}]"
199
200
201     def _format_guard(g: Dict[str, Any]) -> str:
202         """Render the R1 net over-seg promotion-guard verdict (with the strict rule for
contrast)."""
203         basis = "rate" if g["use_rates"] else "count"
204         lines = [f"\n## R1 over-seg promotion guard ({basis}-based, "
205                  f"w_merge={g['weight_merge']}, w_split={g['weight_split']})",
206                  f"  NET verdict: {'PASS' if g['passes'] else 'BLOCK'}  "
```

11. assistant (2026-06-18T05:32:08.269Z)

Now I understand the API. Let me write the verification script that performs the LOO robustness analysis.

12. user (2026-06-18T05:32:09.486Z)

(Bash completed with no output)

13. assistant (2026-06-18T05:32:38.989Z)

```
import os
from dataclasses import replace
from backend.eval import rally_seg_eval as rse, eval_stats
from backend.eval.windowing import SERVED
from backend.eval.tolerance_metrics import over_seg_guard

OUT = "C:/Users/avidu/Projects/badminton-highlight-indexer/output"

golden = [g for g in rse.load_golden(os.path.join(OUT, "human_lovo_manifest.json"), OUT)
          if g["gts"] and os.path.exists(g["trajectory"])]
print("n golden:", len(golden))
names_all = [g["name"] for g in golden]
print("names:", names_all)

baseline = SERVED
milestone = replace(SERVED, max_window_duration=6.0, inactivity_end=1.5,
                    inactivity_rally_thresh=0.20, inactivity_min_density=0.30)
cap12 = replace(SERVED, max_window_duration=12.0)
cap6 = replace(SERVED, max_window_duration=6.0)

def rows(p):
    return {r["name"]: r for r in rse.evaluate(golden, p)["rows"]}

base_rows = rows(baseline)
mile_rows = rows(milestone)
cap12_rows = rows(cap12)
cap6_rows = rows(cap6)

names = [g["name"] for g in golden]  # canonical order
```

---- full-n confirmation of orchestrator numbers ----

```
def paired(metric, A, B, drop=None):
```

```
    use = [n for n in names if n != drop]
    a = [A[n][metric] for n in use]
    b = [B[n][metric] for n in use]
    return eval_stats.paired_delta_ci(a, b), use

print("\n=== FULL n=13 confirmation ===")
d, _ = paired("tol_f1", base_rows, mile_rows)
print("baseline->milestone tolF1:", round(d["mean_delta"],4),
      [round(d["ci_low"],4), round(d["ci_high"],4)],
      "sign", d["sign_test"]["wins"], d["sign_test"]["losses"], d["sign_test"]["ties"],
      "p=", round(d["sign_test"]["p_value"],4), "ci_excl0", d["ci_excludes_zero"])

d2, _ = paired("tol_f1", cap12_rows, cap6_rows)
print("cap12->cap6 tolF1:", round(d2["mean_delta"],4),
      [round(d2["ci_low"],4), round(d2["ci_high"],4)],
      "sign", d2["sign_test"]["wins"], d2["sign_test"]["losses"], d2["sign_test"]["ties"],
      "p=", round(d2["sign_test"]["p_value"],4))

d3, _ = paired("tol_f1", cap6_rows, mile_rows)
print("cap6->+R4 tolF1:", round(d3["mean_delta"],4),
      [round(d3["ci_low"],4), round(d3["ci_high"],4)],
      "sign", d3["sign_test"]["wins"], d3["sign_test"]["losses"], d3["sign_test"]["ties"],
      "p=", round(d3["sign_test"]["p_value"],4))
```

find the ONE loss in cap6->+R4

```
print("\n--- cap6->+R4 per-video tolF1 deltas (find the loss) ---")
for n in names:
    dd = mile_rows[n]["tol_f1"] - cap6_rows[n]["tol_f1"]
    flag = " <== LOSS" if dd < -1e-9 else (" tie" if abs(dd) < 1e-9 else "")
    print(f"  {n}: {cap6_rows[n]['tol_f1']:.4f} -> {mile_rows[n]['tol_f1']:.4f}
d={dd:+.4f}{flag}")
```

mean merge_rate

```
import statistics
mr_base = statistics.mean(base_rows[n]["merge_rate"] for n in names)
mr_mile = statistics.mean(mile_rows[n]["merge_rate"] for n in names)
print(f"\nmean merge_rate baseline->milestone: {mr_base:.3f} -> {mr_mile:.3f}")
g = over_seg_guard([base_rows[n] for n in names], [mile_rows[n] for n in names])
print("over_seg_guard full n:", "passes", g["passes"], "strict", g["strict_passes"],
      "base_net", round(g["base_net"],4), "cand_net", round(g["cand_net"],4),
      "net_delta", round(g["net_delta"],4), "no_merge_rate_regress", g["no_merge_rate_regress"])
```

new clips

```
for nc in ("GX010137", "GX010141"):
    if nc in base_rows:
        print(f"new clip {nc} tolF1: {base_rows[nc]['tol_f1']:.4f} ->
{mile_rows[nc]['tol_f1']:.4f}")
```

---- LOO: baseline->milestone tolF1, drop each clip ----

```
print("\n=== LOO baseline->milestone tolF1 (drop one, n=12) ===")
results = []
for drop in names:
    d, use = paired("tol_f1", base_rows, mile_rows, drop=drop)
    st = d["sign_test"]
    survives = d["ci_excludes_zero"] and (st["p_value"] < 0.05)
    results.append((drop, d["mean_delta"], d["ci_low"], d["ci_high"],
                    st["wins"], st["losses"], st["ties"], st["p_value"],
                    d["ci_excludes_zero"], survives))
```

sort by mean_delta ascending (worst first), but also report by ci_low

```
results_sorted = sorted(results, key=lambda r: r[1])
```

```
print(f"{'drop':<18}{'meanD':>9}{'ci_low':>9}{'ci_hi':>9}{'sign(w/l/t)':>13}{'p':>9}{'ci>0':>7}{
'SURV':>6}")
for r in results_sorted:
    drop, md, lo, hi, w, l, t, p, ce, surv = r
    print(f"{drop:<18}{md:>9.4f}{lo:>9.4f}{hi:>9.4f}{f'{w}/{l}/{t}':>13}{p:>9.4f}{str(ce):>7}{st
r(surv):>6}")

all_survive = all(r[9] for r in results)
print("\nALL 13 LOO drops survive (CI excl 0 AND sign p<0.05):", all_survive)

worst_by_delta = min(results, key=lambda r: r[1])
worst_by_p = max(results, key=lambda r: r[7])
worst_by_cilow = min(results, key=lambda r: r[2])
print("worst by mean_delta:", worst_by_delta[0], "meanD", round(worst_by_delta[1],4),
      "p", round(worst_by_delta[7],4))
print("worst by p_value:   ", worst_by_p[0], "p", round(worst_by_p[7],4),
      "meanD", round(worst_by_p[1],4))
print("worst by ci_low:    ", worst_by_cilow[0], "ci_low", round(worst_by_cilow[2],4))
```

call out the 3 special clips

```
print("\nSpecial-clip drops:")
for sp in ("GX010137", "GX010141", "mahadevpura_1"):
    for r in results:
        if r[0] == sp:
            print(f"  drop {sp}: meanD {r[1]:+.4f} ci [{r[2]:.4f},{r[3]:.4f}] "
                  f"sign {r[4]}/{r[5]}/{r[6]} p={r[7]:.4f} survives={r[9]}")
```

14. user (2026-06-18T05:32:39.012Z)

```
File created successfully at: C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/verify-n13/scratch/lens_loo.py (file state is current in your context
? no need to Read it back)
```

15. user (2026-06-18T05:32:50.385Z)

```
n golden: 13
names: ['adarsh_avi_singles', 'kushagra_singles', 'largetest_doubles', 'gbaaddy',
'testlarge_short', 'mahadevpura_singles', 'mahadevpura_2', 'GX010128', 'mahadevpura_1',
'GX030094', 'Badminton_BXH_2', 'GX010141', 'GX010137']

=== FULL n=13 confirmation ===
baseline->milestone tolF1: 0.2222 [0.1522, 0.2859] sign 12.0 0.0 1.0 p= 0.0005 ci_excl0 True
cap12->cap6 tolF1: 0.1099 [0.0703, 0.1526] sign 12.0 0.0 1.0 p= 0.0005
cap6->+R4 tolF1: 0.0397 [0.0145, 0.0651] sign 9.0 1.0 3.0 p= 0.0215

--- cap6->+R4 per-video tolF1 deltas (find the loss) ---
  adarsh_avi_singles: 0.4957 -> 0.5520  d=+0.0564
  kushagra_singles: 0.0741 -> 0.0779  d=+0.0038
  largetest_doubles: 0.5645 -> 0.5833  d=+0.0188
  gbaaddy: 0.1854 -> 0.2778  d=+0.0923
  testlarge_short: 0.2000 -> 0.2000  d=+0.0000 tie
  mahadevpura_singles: 0.4667 -> 0.5301  d=+0.0635
  mahadevpura_2: 0.1220 -> 0.1220  d=+0.0000 tie
  GX010128: 0.3472 -> 0.4662  d=+0.1189
  mahadevpura_1: 0.0000 -> 0.0000  d=+0.0000 tie
  GX030094: 0.3538 -> 0.4194  d=+0.0655
  Badminton_BXH_2: 0.2273 -> 0.3279  d=+0.1006
  GX010141: 0.3929 -> 0.4314  d=+0.0385
  GX010137: 0.7273 -> 0.6849  d=-0.0423 <== LOSS

mean merge_rate baseline->milestone: 0.651 -> 0.161
over_seg_guard full n: passes True strict False base_net 1.3026 cand_net 0.4289 net_delta
-0.8737 no_merge_rate_regress True
new clip GX010137 tolF1: 0.4478 -> 0.6849
```

```
new clip GX010141 tolF1: 0.3333 -> 0.4314

=== LOO baseline->milestone tolF1 (drop one, n=12) ===
drop                  meanD   ci_low    ci_hi  sign(w/l/t)        p   ci>0  SURV
GX010128             0.2019   0.1474   0.2586 11.0/0.0/1.0   0.0010   True  True
mahadevpura_singles   0.2117   0.1438   0.2801 11.0/0.0/1.0   0.0010   True  True
Badminton_BXH_2      0.2134   0.1436   0.2818 11.0/0.0/1.0   0.0010   True  True
largetest_doubles    0.2169   0.1451   0.2874 11.0/0.0/1.0   0.0010   True  True
GX030094             0.2175   0.1457   0.2859 11.0/0.0/1.0   0.0010   True  True
gbaaddy              0.2176   0.1456   0.2885 11.0/0.0/1.0   0.0010   True  True
GX010137             0.2210   0.1505   0.2899 11.0/0.0/1.0   0.0010   True  True
testlarge_short      0.2241   0.1506   0.2964 11.0/0.0/1.0   0.0010   True  True
adarsh_avi_singles   0.2267   0.1550   0.2993 11.0/0.0/1.0   0.0010   True  True
mahadevpura_2        0.2306   0.1639   0.2987 11.0/0.0/1.0   0.0010   True  True
GX010141             0.2325   0.1618   0.2990 11.0/0.0/1.0   0.0010   True  True
kushagra_singles     0.2342   0.1654   0.3042 11.0/0.0/1.0   0.0010   True  True
mahadevpura_1        0.2407   0.1775   0.3015 12.0/0.0/0.0   0.0005   True  True

ALL 13 LOO drops survive (CI excl 0 AND sign p<0.05): True
worst by mean_delta: GX010128 meanD 0.2019 p 0.001
worst by p_value:    adarsh_avi_singles p 0.001 meanD 0.2267
worst by ci_low:     Badminton_BXH_2 ci_low 0.1436

Special-clip drops:
  drop GX010137: meanD +0.2210 ci [0.1505,0.2899] sign 11.0/0.0/1.0 p=0.0010 survives=True
  drop GX010141: meanD +0.2325 ci [0.1618,0.2990] sign 11.0/0.0/1.0 p=0.0010 survives=True
  drop mahadevpura_1: meanD +0.2407 ci [0.1775,0.3015] sign 12.0/0.0/0.0 p=0.0005 survives=True
```

16. assistant (2026-06-18T05:32:53.725Z)

```
All numbers reproduce and the LOO analysis is complete. Let me verify the bootstrap CI
determinism by checking the seed default and confirming the worst-case is stable.
```

17. user (2026-06-18T05:33:07.643Z)

```
baseline->milestone per-video deltas:
  adarsh_avi_singles: 0.3834 -> 0.5520 d=+0.1686
  kushagra_singles: 0.0000 -> 0.0779 d=+0.0779
  largetest_doubles: 0.2970 -> 0.5833 d=+0.2863
  gbaaddy: 0.0000 -> 0.2778 d=+0.2778
  testlarge_short: 0.0000 -> 0.2000 d=+0.2000
  mahadevpura_singles: 0.1818 -> 0.5301 d=+0.3483
  mahadevpura_2: 0.0000 -> 0.1220 d=+0.1220
  GX010128: 0.0000 -> 0.4662 d=+0.4662
  mahadevpura_1: 0.0000 -> 0.0000 d=+0.0000 TIE/LOSS
  GX030094: 0.1408 -> 0.4194 d=+0.2785
  Badminton_BXH_2: 0.0000 -> 0.3279 d=+0.3279
  GX010141: 0.3333 -> 0.4314 d=+0.0980
  GX010137: 0.4478 -> 0.6849 d=+0.2372
drop GX010128 seed 0 ci 0.1474 0.2586 excl0 True
drop GX010128 seed 1 ci 0.14 0.2606 excl0 True
drop GX010128 seed 2 ci 0.1414 0.2575 excl0 True
drop GX010128 seed 7 ci 0.1437 0.2603 excl0 True
```

18. assistant (2026-06-18T05:33:15.012Z)

```
Everything is fully confirmed. Let me summarize the verification.

All numbers reproduced exactly:
- Full n=13 baseline?milestone tolF1: ?+0.2222 [0.1522, 0.2859], sign 12/0/1, p=0.0005 ?
confirmed
- cap12?cap6: ?+0.1099, cap6?+R4: ?+0.0397 (sign 9/1/3, p=0.0215) ? confirmed; the single loss
is GX010137 (0.7273?0.6849, d=?0.0423)
- merge_rate 0.651?0.161, over_seg_guard NET PASS (net_delta ?0.8737), no merge_rate regression
```

? confirmed
- New clips both improve (GX010137 0.448?0.685, GX010141 0.333?0.431) ? confirmed

LOO result: all 13 drops survive (CI excludes 0 AND sign p<0.05). The full-n tie is mahadevpura_1 (blob, 0.000?0.000); dropping it makes the sign-test 12/0/0 (p=0.0005, the STRONGEST drop, not weakest). Worst-case drop is GX010128 ? ?+0.2019 [0.1474, 0.2586], sign 11/0/1, p=0.0010, still strongly significant. CI robust across bootstrap seeds. The win is not driven by any single clip.

19. user (2026-06-18T05:33:28.626Z)

Output does not match required schema: root: must have required property 'lens', root: must have required property 'verdict', root: must have required property 'key_numbers', root: must have required property 'discrepancies', root: must have required property 'milestone_still_stands', root: must have required property 'caveats'

20. user (2026-06-18T05:33:49.143Z)

Output does not match required schema: root: must have required property 'lens', root: must have required property 'verdict', root: must have required property 'key_numbers', root: must have required property 'discrepancies', root: must have required property 'milestone_still_stands', root: must have required property 'caveats'

21. user (2026-06-18T05:34:05.010Z)

Output does not match required schema: root: must have required property 'lens', root: must have required property 'verdict', root: must have required property 'key_numbers', root: must have required property 'discrepancies', root: must have required property 'milestone_still_stands', root: must have required property 'caveats'

22. assistant (2026-06-18T05:34:07.709Z)

The tool call needs the input fields populated. Let me provide them explicitly.

23. user (2026-06-18T05:34:21.856Z)

Output does not match required schema: root: must have required property 'lens', root: must have required property 'verdict', root: must have required property 'key_numbers', root: must have required property 'discrepancies', root: must have required property 'milestone_still_stands', root: must have required property 'caveats'

24. assistant (2026-06-18T05:34:24.873Z)

The arguments aren't being passed. Let me provide the JSON payload explicitly.

25. user (2026-06-18T05:34:39.212Z)

Output does not match required schema: root: must have required property 'lens', root: must have required property 'verdict', root: must have required property 'key_numbers', root: must have required property 'discrepancies', root: must have required property 'milestone_still_stands', root: must have required property 'caveats'

26. assistant (2026-06-18T05:34:46.694Z)

I need to include the parameters in the tool call. Let me retry with the full structured payload.

27. user (2026-06-18T05:35:02.386Z)

Output does not match required schema: root: must have required property 'lens', root: must have required property 'verdict', root: must have required property 'key_numbers', root: must have required property 'discrepancies', root: must have required property 'milestone_still_stands', root: must have required property 'caveats'

28. user (2026-06-18T05:35:23.602Z)

Structured output provided successfully

29. assistant (2026-06-18T05:35:27.786Z)

Verification complete. The R1 milestone verdict is robust to leave-one-video-out at n=13: all 13 drops survive (CI excludes 0 AND sign p<0.05), the worst-case drop is GX010128 (?+0.2019, p=0.0010), and dropping the blob clip mahadevpura_1 actually strengthens the verdict to 12/0/0 (p=0.0005). The win is not driven by any single clip. All orchestrator numbers reproduced exactly with zero discrepancies. The milestone still stands.